

Reduzindo a dependência da Mastersheet

Um roadmap para que o dashboard use cada vez mais os dados que os próprios motoristas geram e as fontes originais — em vez de uma planilha mantida à mão.

Documento de planejamento · 26 de maio de 2026

O diagnóstico abaixo separa tudo que o sistema lê hoje em três tipos de fonte. Essa separação é a chave de todo o roadmap: a maior parte do que é frágil (manual) é, na verdade, redundante — dá para recalcular a partir do que a gente já coleta.

Diagnóstico — o que a Mastersheet entrega hoje

Tipo de fonte	O que é	Exemplos nas abas atuais	Confiabilidade
● First-party	Dados que o próprio app / motoristas produzem	Driver Daily Check-in (check-in e checkout: TKM mapeado, KM dirigido, área, GPS), PMO Notes , Argentina Cash , Assets , Cash	É a verdade na origem. Já é nosso.
● Externa autoritativa	Sistema de terceiro que tem API própria	Ramp DATABASE (Ramp tem API), VID Monthly CALENDAR / DRIVER CALENDAR (Google Calendar), produção/QC do Street View (Google)	Confiável, mas hoje entra por cópia/export manual.
● Manual / derivada	Alguém digita ou calcula à mão na planilha	DASHBOARD (HM) (KM Driven, TKM, Mapping Days, efficiency, breakeven, salário, custo), CTS Goal Management , HR and Vendors Database	É o que mais quebra: aba renomeada, valor desatualizado, erro de digitação.

O ponto central: a maior parte do ● vermelho é redundante. Exemplo concreto que já existe no código: o checkout grava TKM Mapped e Total KM Driven por dia, por motorista. Ou seja, "KM Driven" e "Mapping Days" da DASHBOARD (HM) são só uma soma do que os drivers já reportaram.

PRINCÍPIO DO ROADMAP

Cada campo do dashboard deve subir na escada de fontes: ● manual → ● API da fonte → ● derivado do dado que a gente mesmo coleta. Quanto mais alto, menos a Mastersheet importa.

As fases

Fase 0 — Tornar a dependência visível

1–2 dias · risco zero

Não dá para reduzir o que não se mede. Antes de migrar nada:

- **Mapa de proveniência:** uma tabela (num `DATA_SOURCES.md` ou comentário no `Code.gs`) listando cada campo do dashboard → qual aba/coluna → tipo ● ● ●.
- **Métrica de progresso:** "% de campos que não tocam aba manual". A meta de cada fase é subir esse número.

Fase 1 — Recalcular produção do checkout, não da DASHBOARD (HM) MAIOR GANHO

é por aqui que se começa · menor risco, maior impacto

A `DASHBOARD (HM)` é a aba ● mais editada. Este é o golpe que mais reduz dependência.

- Criar endpoint `getProductionFromCheckins(startDate, endDate)`: agrega a aba de check-in e calcula, por motorista, a soma de TKM e KM, conta de dias com checkout (= **Mapping Days**) e efficiency (TKM/KM).
- O dashboard passa a ler **essas** métricas; a `DASHBOARD (HM)` vira fallback/conferência.
- **Validação paralela primeiro:** por 1–2 semanas, mostrar derivado vs. planilha lado a lado. Onde bater, desliga a leitura da planilha para aquele campo.

Fase 2 — Cadastro mestre de motorista

isolar e blindar, não eliminar

`HR and Vendors Database` é o único ● que **não** dá para derivar — quem é motorista, email, país, casa e status são cadastro de verdade.

- Eleger **uma** aba como cadastro canônico; todo o resto referencia por `email` (já é a chave hoje). Nada de duplicar nome/país em outras abas.
- Endpoint único `getDriverRegistry()` cacheado, usado por todas as páginas.
- Médio prazo: é o primeiro candidato a sair do Sheets para um store de verdade — pequeno, estável e crítico.

Fase 3 — Fontes externas direto da API

cortar a cópia manual entre sistemas

Onde hoje alguém exporta/cola dados de terceiro numa aba, trocar pela API:

- **Ramp:** API REST → busca transações direto (token no `PropertiesService`, nunca hardcoded) → mata a aba `Ramp DATABASE`.
- **Calendário (VID / Driver):** Google Calendar API → escala/dias vêm do calendário real, não de cópia mensal.
- **Produção / QC do Street View:** via API da Google onde houver; onde não houver, o checkout do motorista (●) é o proxy.

Regra: cada fonte entra **uma vez**, via API, cacheada. A planilha deixa de ser o "pen drive" entre sistemas.

Fase 4 — Decisão de arquitetura: Sheets continua sendo o banco?

uma bifurcação a escolher

Caminho A — Sheets saneado. Mantém Apps Script + Sheets, mas as abas viram tabelas limpas só de escrita do app (check-in, registry, notes), sem nada manual/derivado. Barato, zero migração, combina com "ferramenta interna sem build step". Limite: performance quando o check-in crescer muito.

Caminho B — Store real por baixo. Move o dado first-party para um banco leve (Firestore, Supabase/Postgres). O Sheets vira só uma view exportada. Mais robusto e rápido, mas é projeto de verdade (auth, deploy, backup) e foge do stack atual.

Para o tamanho de hoje, **A resolve ~90%** e é o que eu perseguiria primeiro. B só quando volume/latência do check-in começar a doer.

Resumo — a escada, em ordem de execução

- 1 **Fase 0** — mapa de proveniência + métrica de % (mede o problema)
- 2 **Fase 1** — produção (KM / TKM / dias) do checkout, não da `DASHBOARD (HM)`
- 3 **Fase 2** — cadastro único de motorista por `email`
- 4 **Fase 3** — Ramp / Calendar / Street View via API, fim das cópias manuais

5 **Fase 4** — decidir Sheets-saneado (A) vs. banco real (B)

Cada fase deixa o sistema inteiro funcionando — nada de big bang.